SOFTWARE ENGINEERING MINI-PROJECT · TEAM H1

Test Plan — Simple Shell

Simple Shell — C11 / POSIX command-line interpreter

Document ID	H1-TP-001
Version	2.0
Date	12 November 2026
Status	Final
Standard	ISO/IEC/IEEE 29119-3:2021, following the IEEE 829 test-plan structure
Team	H1 — Simple Shell
Repository	https://github.com/SE-H1-SimpleShell/MiniProject_H1_SimpleShell
Related documents	`docs/SRS.md`, `docs/RTM.md`, `docs/Test_Cases.md`, `docs/Validation_Specification.md`, `docs/Test_Report.md`

Table of Contents

Document Control
1. Introduction
    1.1 Purpose
    1.2 Scope
    1.3 References
    1.4 Definitions and Abbreviations
    1.5 Document Overview
2. Test Items and Features
    2.1 Test Items
    2.2 Features to Be Tested
    2.3 Features Not to Be Tested
    2.4 Test Environment
3. Test Approach
    3.1 Test Levels and Methods
    3.2 Test Techniques and Coverage
    3.3 Test Procedure and Commands
    3.4 Test Data and Fixtures
    3.5 Defect Lifecycle and Retest
    3.6 Roles and Responsibilities
4. Pass/Fail Criteria
    4.1 Entry Criteria
    4.2 Pass/Fail Criteria by Test Level
    4.3 Suspension and Resumption Criteria
    4.4 Exit Criteria
    4.5 Metrics and Reporting
    4.6 Risks and Mitigations
5. Test Deliverables and Security Validation
    5.1 Security Validation
    5.2 Test Deliverables
    5.3 Traceability to the SRS
    5.4 Reporting and Evidence Retention
    5.5 Approval

Document Control

Version	Date	Author	Change
1.0	Phase 1	Team H1	Initial test plan
2.0	12 November 2026	Team H1	Restructured to IEEE 829/29119-3; Sections 3–5 expanded and security validation added as Section 5.1

1. Introduction

1.1 Purpose

This plan defines how Simple Shell is verified against the requirements in `docs/SRS.md`. It identifies the test items, the features to be tested and excluded, the test approach, the pass/fail criteria, the security validation, and the deliverables and evidence that must be retained.

1.2 Scope

In scope: interactive input and termination; built-in commands; external commands; whitespace, quotes, escapes, and operators; input/output redirection; pipelines (2 to 32 stages); background execution and asynchronous reaping; malformed input; missing commands and files; process and resource handling; security validation of the parser boundary; CI and sanitizer checks.

Out of scope: full POSIX shell scripting, variable expansion, globbing, command substitution, aliases, full job control, remote services, and tests against systems not owned by the team.

1.3 References

Reference	Use
`docs/SRS.md`	Requirements baseline (FR, NFR, SO, SR)
`docs/RTM.md`	Requirement-to-test traceability
`docs/Test_Cases.md`	TC-01 … TC-18 and NFRV-01 … NFRV-08
`docs/Validation_Specification.md`	Security validation SEC-01 … SEC-06 and penetration-test plan
`docs/Test_Report.md`	Recorded execution evidence
`docs/Resource_Audit.md`	Memory, descriptor, and termination audit evidence
`.github/workflows/ci.yml`	Automated CI definition
ISO/IEC/IEEE 29119-3:2021	Test documentation structure
IEEE 829-2008	Test-plan clause structure retained for course alignment

1.4 Definitions and Abbreviations

Term	Meaning
FR / NFR	Functional / non-functional requirement
SO / SR	Security objective / security requirement
TC	Functional test case
NFRV	Non-functional verification procedure
SEC	Security validation activity
ASan / UBSan	AddressSanitizer / UndefinedBehaviorSanitizer
SUT	System under test — the `simple-shell` binary
CI	Continuous integration (GitHub Actions)

1.5 Document Overview

Sections 2–5 follow the IEEE 829 outline: test items and features, approach, pass/fail criteria, and deliverables. Section 5.1 is dedicated to security validation.

2. Test Items and Features

2.1 Test Items

Item	Description	Location
`simple-shell`	Interactive shell binary built from `src/*.c`	Build output
Parser unit harness	Direct tests of `parse_line()` and `free_pipeline()`	`tests/test_parser.c`
Executor harness	Direct tests of `execute_pipeline()` and child handling	`tests/test_executor.c`
Shell integration harness	End-to-end command scenarios through the binary	`tests/test_shell.sh`
Sanitizer build	ASan/UBSan build of the same binaries	`make asan-test`
Valgrind audit	Memory and descriptor checks under Valgrind	`make valgrind`
CI workflow	Build, test, sanitizer, Valgrind, and cleanliness jobs	`.github/workflows/ci.yml`

2.2 Features to Be Tested

Feature	Requirements	Test coverage
Interactive prompt, EOF, and `exit`	FR-01, FR-10	TC-01, TC-10
External command execution and status	FR-02, FR-09	TC-02, TC-16
Built-ins (`cd`, `pwd`, `echo`, `help`, `exit`)	FR-03	TC-03, TC-04, TC-18
Tokenisation, quotes, escapes, operator validation	FR-04	TC-05, TC-06, TC-11
Input/output redirection	FR-05	TC-07, TC-08, TC-09
Pipelines, including built-in stages and 32-stage limit	FR-06	TC-12
Background execution and reaping	FR-07	TC-13
Diagnostics and recovery from malformed input	FR-08	TC-11, TC-14, TC-15
Reliability and stress behaviour	NFR-01	NFRV-01, TC-17
Module/header separation	NFR-02	NFRV-02
Memory and descriptor hygiene	NFR-03	NFRV-03
Prompt and diagnostic usability	NFR-04	NFRV-04
Linux/GCC C11 portability	NFR-05	NFRV-05
CI testability	NFR-06	NFRV-06
Security hygiene and injection resistance	NFR-07, SR-01, SR-02	NFRV-07, SEC-01, SEC-02, SEC-05
Documentation consistency	NFR-08	NFRV-08
Safe failure and recovery	SR-04	SEC-03, SEC-04, TC-14, TC-15
Sanitizer cleanliness	SR-03	SEC-06, NFRV-03

2.3 Features Not to Be Tested

Full POSIX shell compliance, shell variables, glob expansion, command substitution, aliases, scripting, terminal job control, and network behaviour are out of scope. Penetration testing is planned but not executed in this phase; see Section 5.1.4.

2.4 Test Environment

Linux (x86_64 or the agreed team VM), GCC in C11 mode, GNU Make, Bash, GitHub Actions (`ubuntu-latest`), AddressSanitizer/UndefinedBehaviorSanitizer, and Valgrind. Record the OS and compiler versions, commit SHA, commands, and complete output for every reported run.

3. Test Approach

3.1 Test Levels and Methods

Level	Method	Evidence
Unit/component	Parser and direct executor harnesses	Parser and executor output
Integration	Shell command scenarios through `tests/test_shell.sh`	Actual integration-test output
Robustness/security	Malformed input, missing files, limits, repeated runs	Inputs, outputs, and sanitizer diagnostics
System/acceptance	Fresh clone, build, and command demonstration	Test report and demo evidence
CI	GitHub Actions on push and pull request	Workflow URL and status

3.2 Test Techniques and Coverage

- **Boundary-value analysis:** argument count at 128/129, pipeline stages at 32/33, descriptor limit at 64.
- **Equivalence partitioning:** valid versus malformed operators, valid versus missing files, foreground versus background commands, built-in versus external stages.
- **Error-guessing and negative testing:** unterminated quotes, trailing backslash, leading or trailing `|`, duplicate redirections, non-final `&`.
- **Concurrency and deadlock testing:** large data streams (`seq 50000 | cat | wc -l`) and early consumer exit (`yes | cat | head -n 1`).
- **State and side-effect checks:** rejected syntax must not create or truncate files, change directories, or spawn processes.
- **Sanitizer and Valgrind runs:** repeat the functional suites under ASan/UBSan and the unit harnesses under Valgrind.

3.3 Test Procedure and Commands

From a clean checkout:

```
make clean
make
make test
make asan-test
```

`make test` runs the parser harness, executor harness, and shell integration suite. `make asan-test` rebuilds with `-fsanitize=address,undefined` and repeats the same tests. Use `make asan-test`, not `make asan && make test`: `make asan` only produces a sanitized binary, and the following `make test` would rebuild without sanitizer flags, so nothing would actually be validated under ASan or UBSan.

The complete local sequence used for release evidence is:

```
make clean && make all && make test && make asan-test && make valgrind && make ci
```

`make ci` runs the consolidated build, functional, sanitizer, and cleanliness targets. The same sequence is executed by `.github/workflows/ci.yml` on every push and pull request.

3.4 Test Data and Fixtures

Fixture	Purpose
Temporary files under `/tmp/simple-shell-test-*`	Redirection input, overwrite, and append checks
`/dev/null` and a missing path	Input-redirection success and failure checks
Generated streams (`seq`, `yes`)	Pipeline deadlock and SIGPIPE checks
Repeated 2-stage and 4-stage pipelines under `ulimit -n 64`	Descriptor-leak checks
Malformed command strings	Parser rejection and side-effect checks

Fixtures are created and removed by the harnesses; no test depends on files committed to the repository outside `tests/`.

3.5 Defect Lifecycle and Retest

Reproduce → link to requirement/test and issue → patch on a feature branch → add a regression test → run focused and regression tests → open PR → review and CI → merge → record retest evidence.

3.6 Roles and Responsibilities

Role	Responsibility
Test owner	Maintains this plan, test cases, and RTM coverage
Developers	Add regression tests with each fix and keep `make test` green
Reviewers	Verify the test evidence in the PR and on CI
CI	Executes the build, test, sanitizer, Valgrind, and cleanliness jobs

4. Pass/Fail Criteria

4.1 Entry Criteria

- Source, headers, and tests are checked out at the target commit.
- GCC, Make, Bash, and (for the audit target) Valgrind are available.
- The target commit SHA and expected results are identified.
- The test environment matches Section 2.4.

4.2 Pass/Fail Criteria by Test Level

Level	Pass condition	Fail condition
Unit/component	All parser and executor assertions pass	Any assertion fails, crashes, or hangs
Integration	All shell scenarios return the expected output and exit status	Wrong output, unexpected side effect, or hang
Robustness/security	Malformed input is rejected with a diagnostic; no file or process side effect; no sanitizer finding	Crash, leak, sanitizer report, or side effect
Sanitizer	ASan/UBSan report zero errors and zero leaks	Any sanitizer diagnostic
Valgrind	No invalid memory access and no leaked parser memory	Valgrind error or leak
CI	All jobs succeed on the target commit	Any job fails or is skipped without justification

Any test that exceeds the harness timeout (30 seconds for the executor pipeline alarm; 15 minutes for the CI job) is recorded as a failure, not retried silently.

4.3 Suspension and Resumption Criteria

Suspend testing when:

- The build does not compile on the target environment.
- A blocking defect prevents the suite from completing.
- CI infrastructure is unavailable for more than one working day.
- The target commit or environment changes during a run.

Resume when:

- The blocking fix is merged and CI is green.
- The environment is restored and the commit SHA is re-identified.
- The affected tests are re-run from a clean checkout.

4.4 Exit Criteria

- Automated tests are run and outcomes are recorded.
- Failures and deviations are explained, fixed, and re-tested.
- Manual coverage gaps are exercised or documented as unresolved.
- Sanitizer output and the test report reflect real evidence.
- Every FR has at least one executed test case and every NFR has an executed verification procedure.
- Security validation SEC-01 … SEC-06 has recorded evidence, with any untested technique listed as a limitation.

4.5 Metrics and Reporting

Metric	Target	Source
Functional test case pass rate	100% of executed cases	`docs/Test_Report.md`
Sanitizer findings	0	`make asan-test` output
Valgrind findings	0	`make valgrind` output
Hang/timeout failures	0	Harness logs
Requirements covered	100% of FR and NFR	`docs/RTM.md`
CI success on target commit	All jobs pass	GitHub Actions run

Results are written to `docs/Test_Report.md`; the RTM coverage column is updated in the same change set when a test is added or removed.

4.6 Risks and Mitigations

Risk	Impact	Mitigation
Flaky timing assertion under sanitizers	False failure	Record the run honestly, re-run the full suite, and keep the timing assertion bounded
Environment differences (WSL vs CI)	Non-reproducible result	Record OS, compiler, and runner; treat CI as the release gate
Long-running or hanging pipelines	Blocked suite	30-second alarm in the executor harness and CI timeouts
Descriptor exhaustion	Misleading pass	Repeat pipelines under `ulimit -n 64` and compare descriptor counts
Incomplete system-call fault injection	Unverified error paths	Document the gap in `docs/RTM.md` Section 4 and retain it as a known limitation

5. Test Deliverables and Security Validation

5.1 Security Validation

5.1.1 Security Scope and Threat Model

Security validation focuses on the parser because it processes untrusted input. The executor is a secondary boundary: it forwards already validated tokens to `fork`/`execvp`. The threat model assumes a local user who can type arbitrary text, redirect files they own, and stress resource limits. Privilege escalation, persistence, network attacks, and attacks on other systems are out of scope.

5.1.2 Security Validation Activities

ID	Activity	Pass condition
SEC-01	Malformed syntax: unterminated quotes, trailing backslash, leading/trailing pipe, non-final `&`, duplicate redirections	Each is rejected with an error and the shell stays usable
SEC-02	Argument and segment exhaustion beyond the documented limits	Rejected cleanly with no overflow or leak
SEC-03	Missing and unreadable files in redirections	Error reported, shell remains usable
SEC-04	Non-existent command and non-executable targets	Error reported, shell remains usable
SEC-05	Source review of external execution	`fork` + `execvp` used; no `system()`, no unintended second shell layer
SEC-06	Sanitizer run of the malformed-input suite	No memory or undefined-behaviour findings

These activities map to security requirements SR-01 … SR-04 (SRS Section 3.4.2) and to the verification procedures NFRV-03 and NFRV-07.

5.1.3 Security Pass/Fail Criteria

- **Pass:** every SEC activity produces the documented safe outcome; no file or process side effect occurs before rejection; no sanitizer finding; source review finds no `system(` call.
- **Fail:** a malformed command crashes or terminates the shell, creates or truncates a file, spawns a process before validation, or produces an ASan/UBSan diagnostic.
- **Not tested:** penetration-style fuzzing is not executed in this phase and is reported as a limitation, not as a pass.

5.1.4 Penetration Testing Status

Penetration testing is **planned only**. `docs/Validation_Specification.md` Section 8 defines the authorisation, scope, threat model, eight planned techniques (PEN-01 … PEN-08), tooling (AFL++/libFuzzer, ASan/UBSan, Valgrind), and explicit prohibitions. No penetration test is claimed as executed here.

5.2 Test Deliverables

Deliverable	Location	Status
Test Plan (this document)	`2-Test-Planning/Test_Plan.md`	Final
Test Cases and NFR procedures	`2-Test-Planning/Test_Cases.md`, `docs/Test_Cases.md`	Final
Validation and security specification	`2-Test-Planning/Validation_Specification.md`	Final
Requirements Traceability Matrix	`docs/RTM.md`	Maintained
Test Execution Report	`docs/Test_Report.md`	Recorded
Resource audit report	`docs/Resource_Audit.md`	Recorded
CI run logs and summaries	GitHub Actions `CI` workflow	Available
Demo and acceptance evidence	`9-Demo/Demo_Evidence_Index.md`	Maintained

5.3 Traceability to the SRS

SRS requirements	Test coverage	Evidence
FR-01 … FR-10	TC-01 … TC-18	`docs/Test_Report.md`, CI logs
NFR-01 … NFR-08	NFRV-01 … NFRV-08	`docs/Test_Report.md`, `docs/Resource_Audit.md`
SR-01 … SR-04, SO-01 … SO-03	SEC-01 … SEC-06	`docs/Validation_Specification.md` Section 7, sanitizer and source-review evidence

The canonical row-by-row mapping is `docs/RTM.md`. The functional suite currently defines 18 test cases and the non-functional suite defines 8 verification procedures, exceeding the 7–10 test-case minimum while covering every FR and NFR.

5.4 Reporting and Evidence Retention

Every run records the commit SHA, operating system, compiler version, command, and complete output. The Test Report retains failed or flaky runs together with their resolution; expected output is never presented as observed output. CI logs are retained by GitHub Actions for the repository retention period, and the canonical summaries are committed under `docs/`.

5.5 Approval

Role	Name	Status
Prepared by	Team H1	Complete
Reviewed by	Team members	Complete
Final reviewer	Project guide/instructor	Pending course review